# Chủ đề 9: Lập trình giao diện với Compose và View

**Nội dung chính:**

**9.1.** Tổng quan về Jetpack Compose.

**9.2.** Tạo giao diện khai báo với Compose.

**9.3.** Quản lý trạng thái UI với Compose.

**Mục tiêu cần đạt được:**

**Hiểu** cách làm việc với Jetpack Compose và lợi ích của lập trình UI theo hướng khai báo.

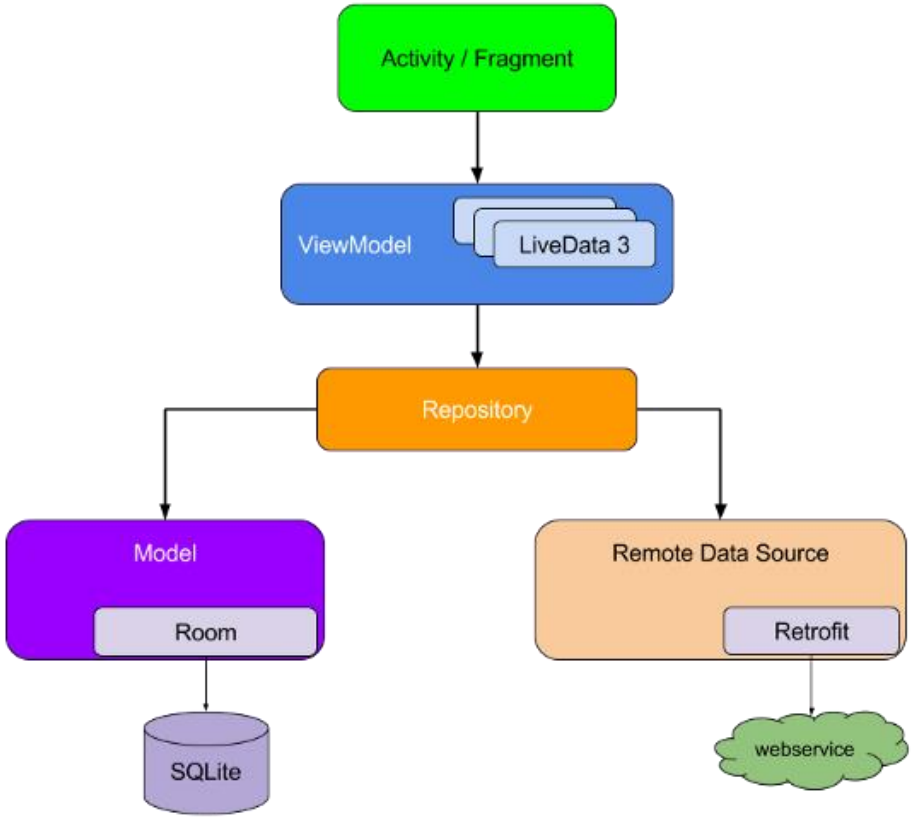
**Biết** cách tạo các thành phần giao diện cơ bản như Text, Button, Row, và Column.

**Vận dụng** các công cụ của Compose để xây dựng giao diện hiện đại, dễ bảo trì.

# Buổi 9: Lập trình theo hướng phản ứng và khung hiển thị

Lập trình theo hướng phản ứng và khung hiển thị với Jetpack Compose.

Học cách xây dựng giao diện di động dễ dàng và hiệu quả với Compose.



# Phương pháp lập trình cho thiết bị di động

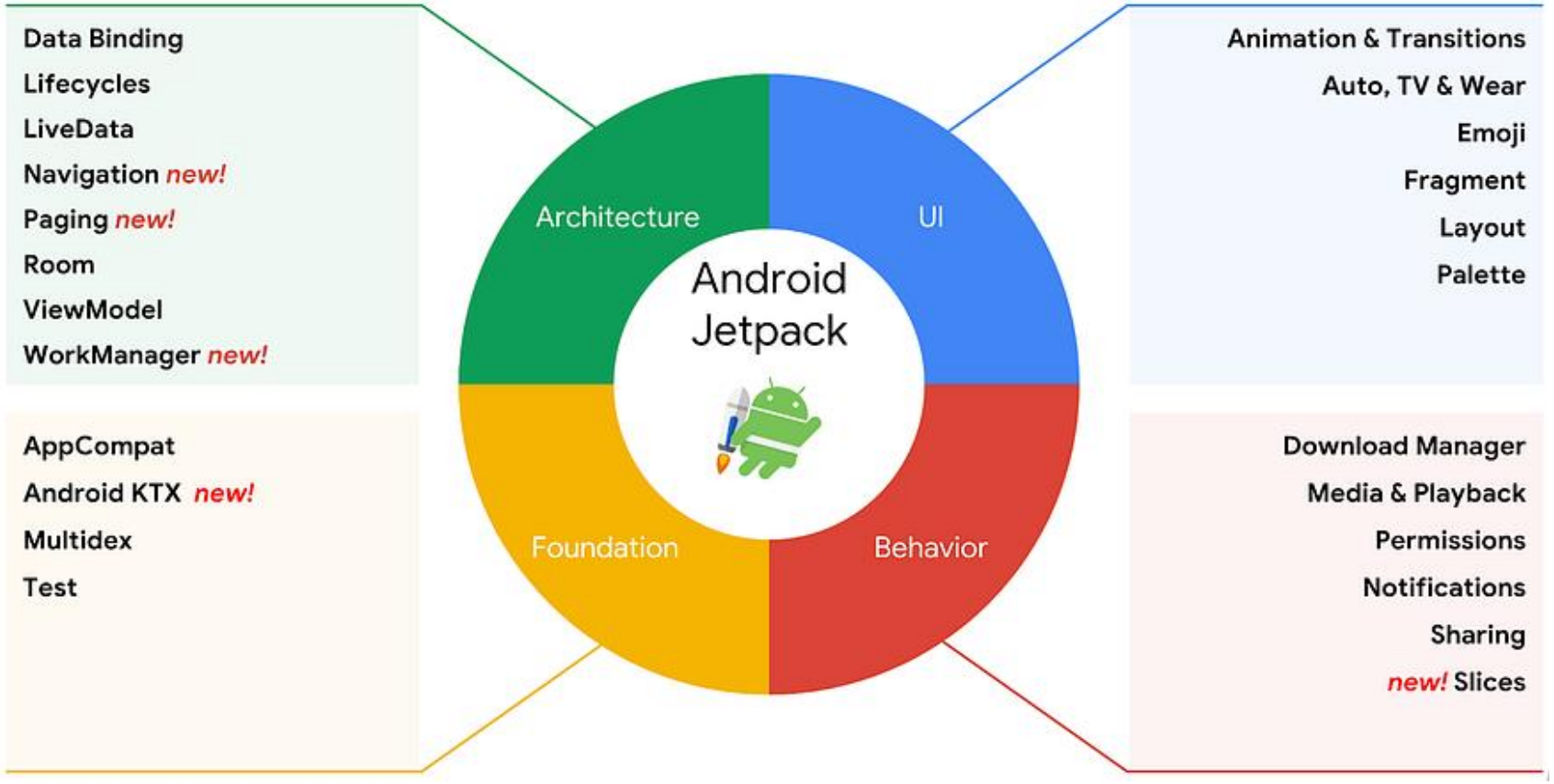
- **Lập trình theo hướng đối tượng (OOP):** Sử dụng đối tượng và lớp.
  - Đặc điểm: Tạo lớp với thuộc tính và phương thức.
  - Ví dụ: Tạo đối tượng Button với các thuộc tính như màu sắc, kích thước.
- **Lập trình theo hướng sự kiện:** Xử lý các sự kiện người dùng như nhấn nút, cuộn màn hình.
  - Ví dụ: Sử dụng setOnClickListener để xử lý sự kiện nhấn nút.
- **Lập trình hướng phản ứng (Reactive Programming):** Xử lý dữ liệu thay đổi theo thời gian và đồng bộ với giao diện người dùng.
  - Ví dụ: Sử dụng LiveData, State trong Compose để tự động cập nhật giao diện.

# Lập trình hướng phản ứng là gì?

- **Khái niệm:** Lập trình phản ứng giúp xử lý dữ liệu thay đổi trong thời gian thực mà không cần phải cập nhật UI thủ công.
- **Các công cụ hỗ trợ:**
  - RxJava: Xử lý luồng dữ liệu bất đồng bộ.
  - LiveData: Quản lý và phản ánh dữ liệu thay đổi trong UI.
  - Jetpack Compose: Tạo giao diện UI tự động cập nhật khi dữ liệu thay đổi.

# Giới thiệu về Jetpack Compose

- **Jetpack Compose:** Là framework UI giúp phát triển giao diện người dùng mà không cần sử dụng XML.



# Giới thiệu về Jetpack Compose

**So sánh compose với XML:**

| Tiêu chí | XML + View | Jetpack Compose |
|---|---|---|
| Kiểu giao diện | Imperative UI | Declarative UI |
| Cách viết code | XML + Java/Kotlin | Kotlin 100% |
| Quản lý state | ViewModel + LiveData | State hoisting, remember |
| Hiệu suất | Có View Hierarchy | Nhẹ hơn, nhanh hơn |

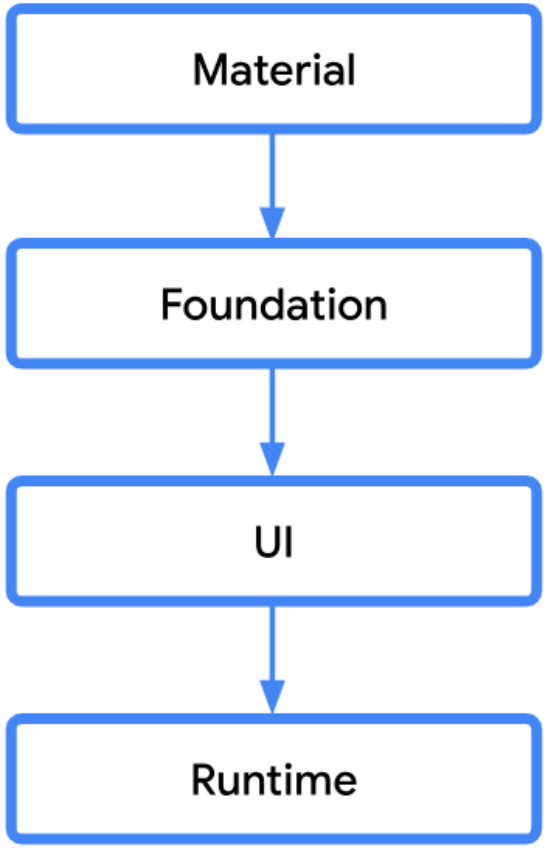
# Giới thiệu về Jetpack Compose

**Jetpack Compose** là môt framework UI khai báo (Declarative UI).

- Giúp làm giao diện nhanh hơn, ít mã nguồn hơn.
- Tích hợp Material Design dễ dàng.
- Hỗ trợ hoàn toàn Kotlin.

**Kiến trúc của Jetpack Compose:**

- **Composable Functions**: Xây dựng UI bằng hàm @Composable.
- **State & Recomposition**: Cập nhật UI linh hoạt.
- **Material Design**: Hỗ trợ giao diện Material mới nhất.
- **Navigation, ViewModel, LiveData**: Tích hợp với các thư viện Jetpack.



# Lợi ích của Jetpack Compose (phần 1)

- **Mã ngắn gọn và dễ hiểu hơn:**

  - Viết giao diện trực tiếp trong mã Kotlin thay vì sử dụng XML.

  - Ví dụ:

```
@Composable
 fun Greeting(name: String) {
     Text(text = "Hello, $name!")
 }
```

- **Quản lý UI đồng bộ:** Giao diện tự động cập nhật khi có sự thay đổi trong dữ liệu.

# Lợi ích của Jetpack Compose (phần 2)

- **Giảm mã boilerplate:** Không cần tạo các tệp XML riêng biệt.
- **Tích hợp dễ dàng với các công cụ Android**: Tích hợp tốt với LiveData, Navigation, ViewModels.
- **Hiệu suất tốt hơn**: Tối ưu hóa việc cập nhật giao diện, giảm thiểu bộ nhớ sử dụng.

# Tổng  quan về giao diện được xây dựng với compose

Mô tả quan hệ giữa các thành phần:

- **@Composable**: Hàm UI khai báo.
- **Modifier**: Tùy chỉnh giao diện (kích thước, padding, click...).
- **Material 3 (M3)**: Các thành phần UI theo Material Design.
- **State & Recomposition**: Cập nhật giao diện linh hoạt.
- **Navigation & ViewModel**: Quản lý dữ liệu và điều hướng.

# Các thành phần cơ bản trong Compose

**@Composable là gì?:** Hàm được đánh dấu @Composable để tạo UI.

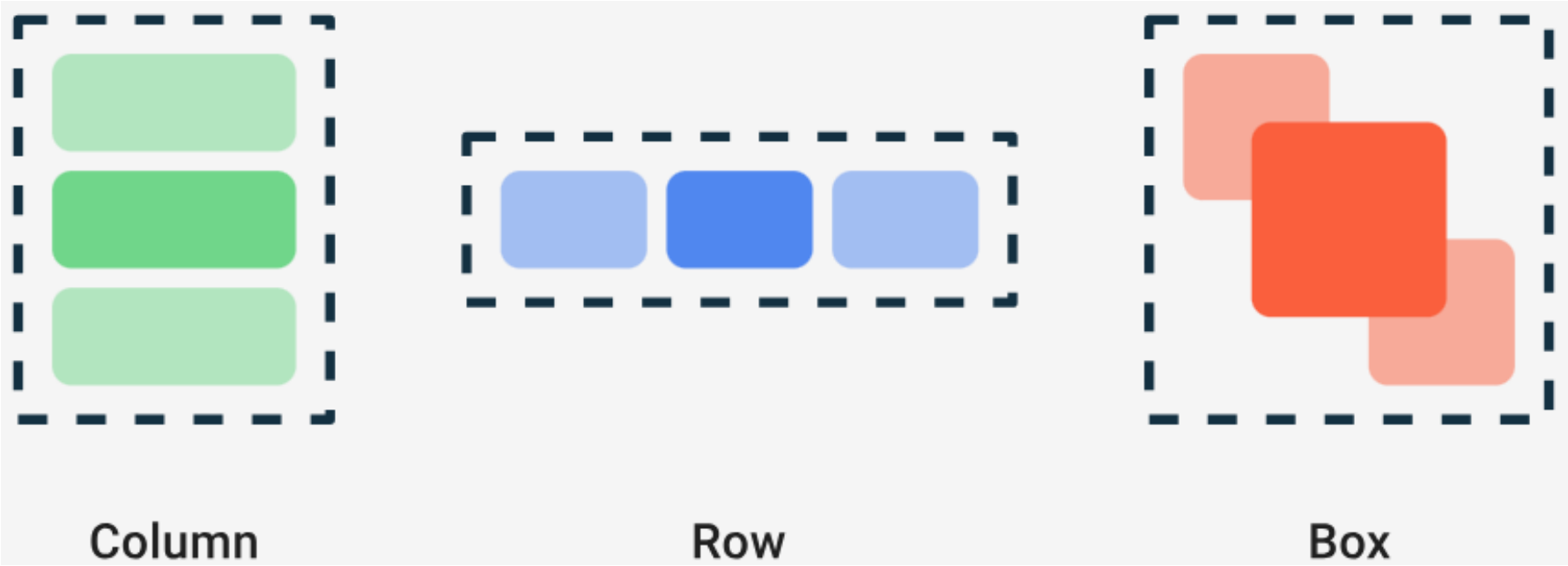
```
@Composable
fun Greeting(name: String) {
    Text(text = "Hello $name")
}
```

**Các thành phần UI cơ bản:** Text(), Button(), Image().

```
Button(onClick = { /*TODO*/ }) {
    Text("Click Me")
}
```

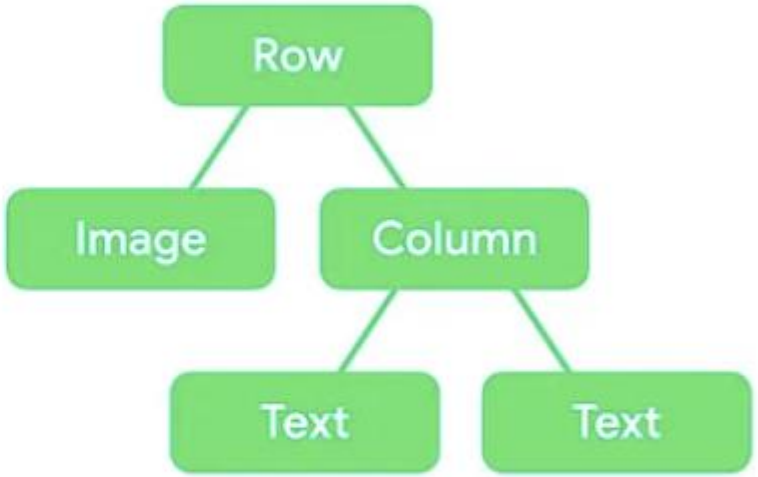
# Bố cục giao diện trong Compose

**Column, Row, Box thay thế LinearLayout, FrameLayout.**

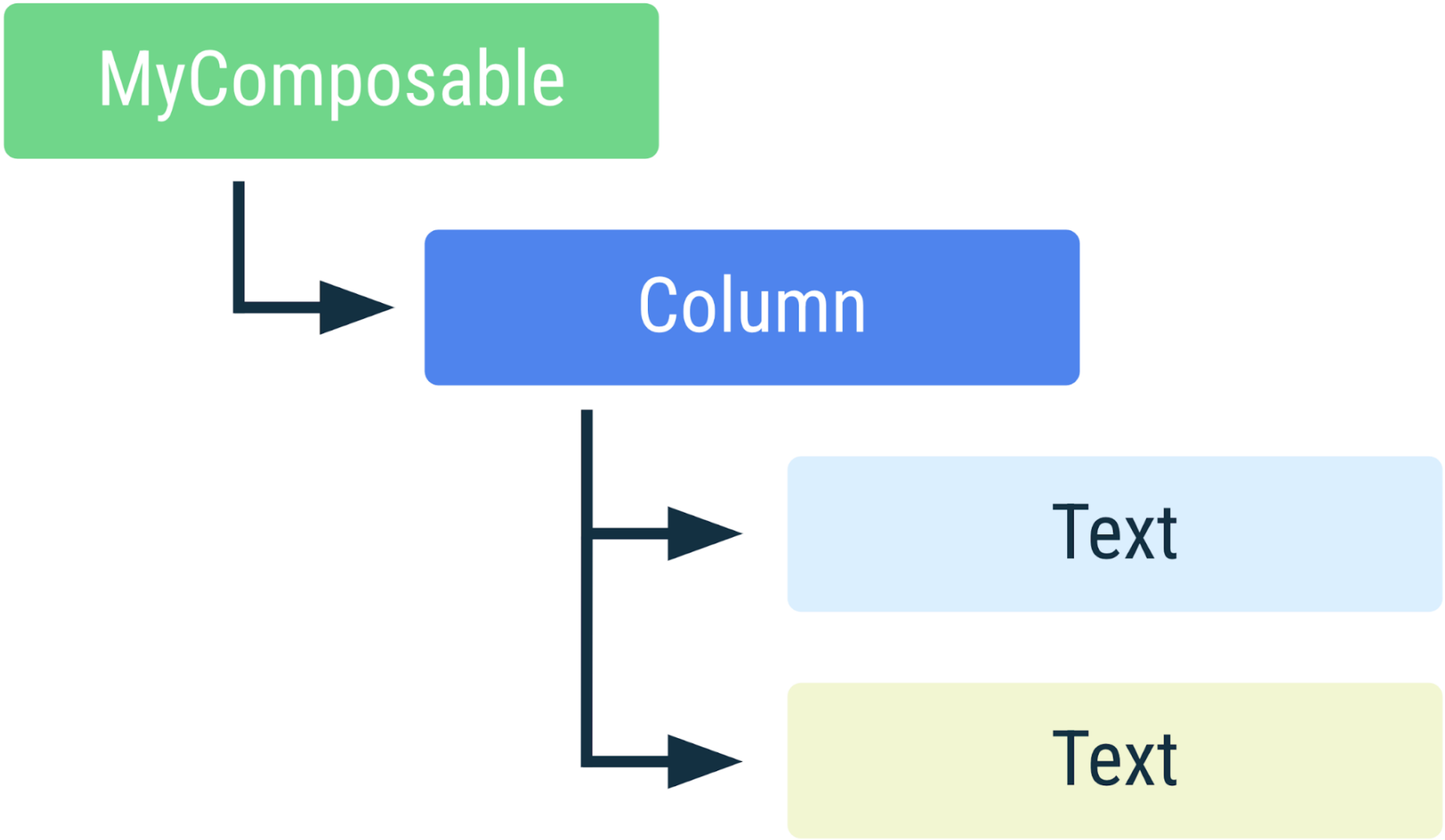


# Bố cục trong Compose

```
Row {
    Image(..)
    Column {
        Text(..)
        Text(..)
    }
}
```



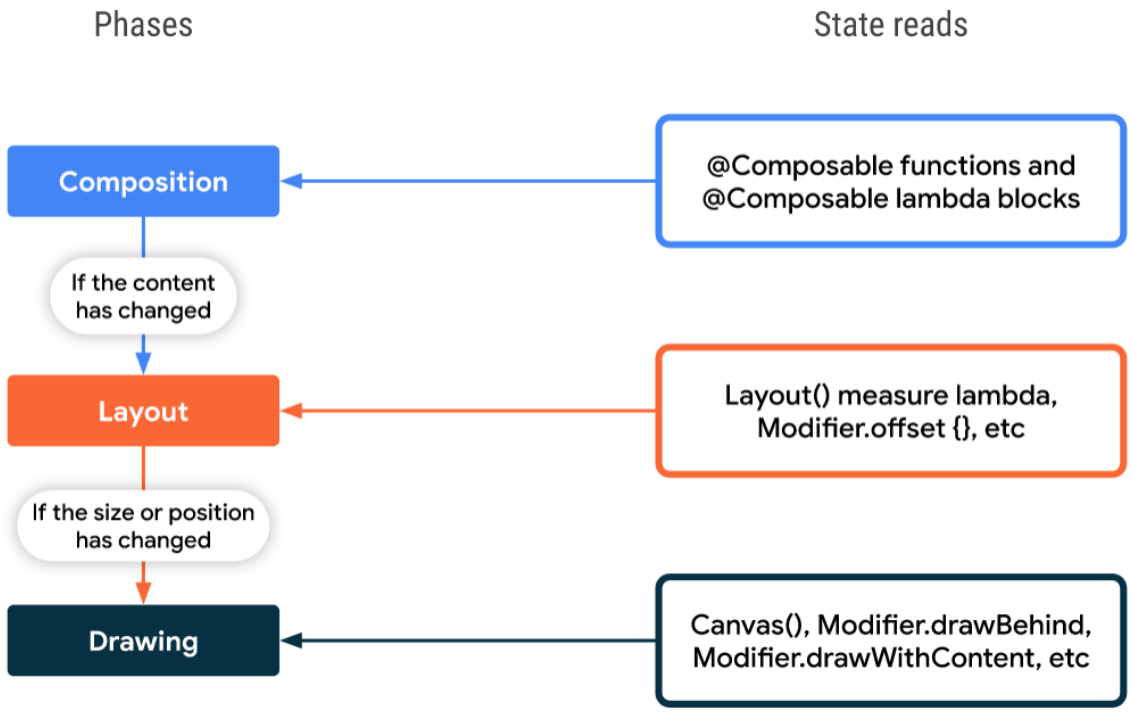
# Bố cục trong Compose



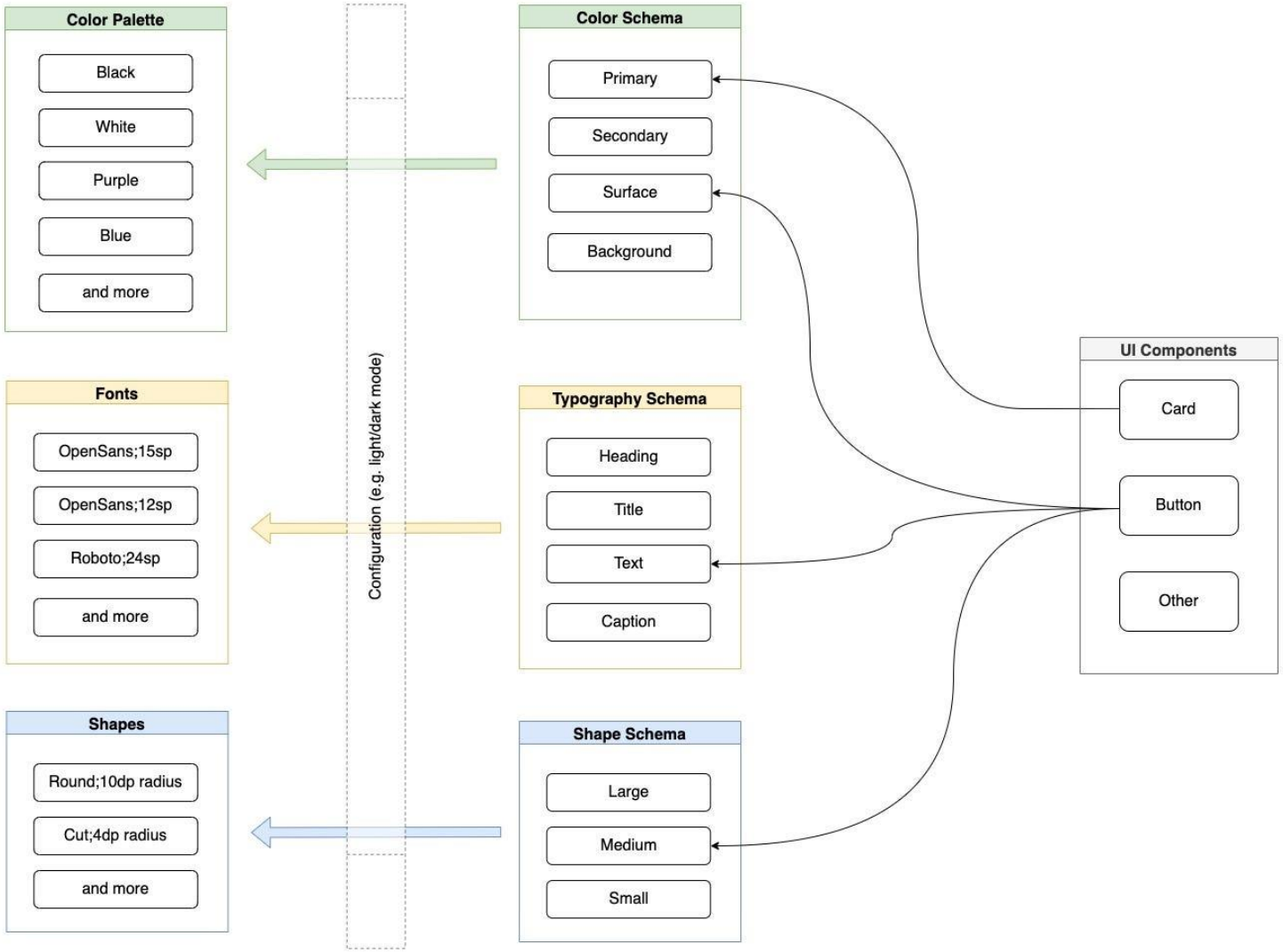
# Các thành phần cơ bản trong Compose

**Modifier - Tuỳ chỉnh giao diệnSử dụng Modifier để cài đặt width, height, padding...**

```
Text(
    text = "Hello",
    modifier = Modifier.padding(16.dp)
)
```



# Tùy chỉnh giao diện với M3



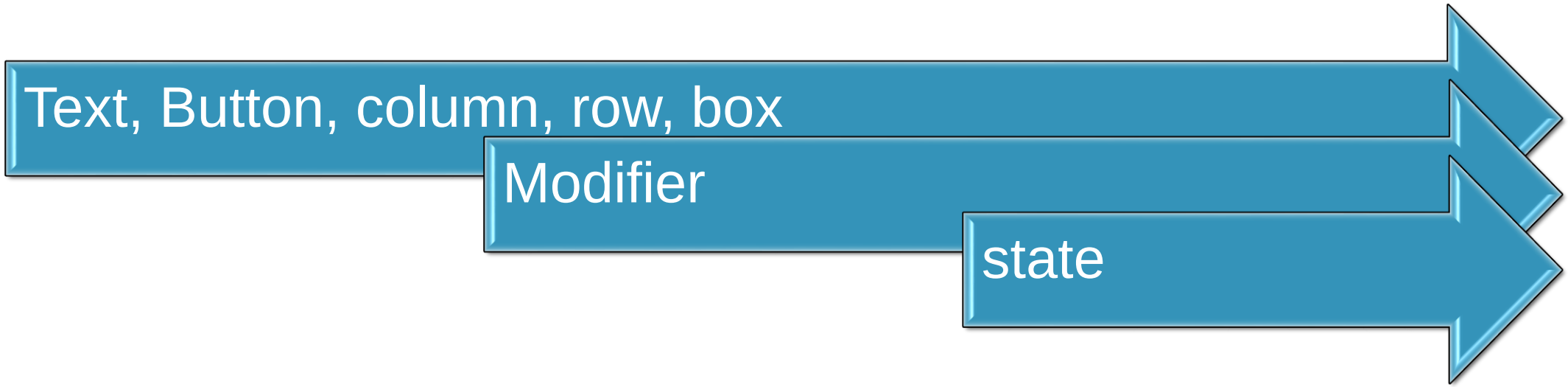
# Quản lý trạng thái

remember để lưu trạng thái UI.

MutableState để cập nhật UI linh hoạt.

```
var count by remember { mutableStateOf(0) }
Button(onClick = { count++ }) {
    Text("Count: $count")
}
```

# Giới thiệu về cách sử dụng các thành phần cơ bản



# Text trong Jetpack Compose - Giới thiệu

- Text: Thành phần cơ bản để hiển thị văn bản trong Compose.
- Tùy chỉnh: Có thể thay đổi kiểu chữ, kích thước, màu sắc.
- Ví dụ:

```
@Composable
 fun SimpleText() {
     Text(text = "Welcome to Compose!")
 }
```

# Text trong Jetpack Compose - Cách sử dụng

- Cách sử dụng: Dùng hàm Text để hiển thị văn bản.
- Tùy chỉnh thêm: Căn chỉnh, thay đổi màu sắc, kích thước.
- Ví dụ:

```
@Composable
fun CustomText() {
    Text(
        text = "Styled Text",           // Nội dung hiển thị
        fontSize = 24.sp,               // Cỡ chữ 24sp
        color = Color.Blue,             // Màu chữ xanh
        modifier = Modifier.padding(16.dp) // Khoảng cách padding 16dp
    )
}
```

# Button trong Jetpack Compose - Giới thiệu

- Button là thành phần để tạo nút bấm, người dùng có thể tương tác với giao diện.
- Tính năng: Dễ dàng thêm các hành động khi người dùng nhấn nút.
- Ví dụ: Tạo nút bấm cơ bản để thay đổi văn bản khi nhấn nút.

# Button trong Jetpack Compose - Cách sử dụng

- Cách sử dụng: Dùng hàm Button để tạo nút, bạn có thể xử lý sự kiện nhấn nút trong hàm onClick.
- Ví dụ:

```
@Composable
fun SimpleButton() {
    Button(onClick = { /* Xử lý sự kiện khi nhấn nút */ }) {
        Text("Click Me!") // Nội dung hiển thị trên nút
    }
}
```

# Button trong Jetpack Compose - Ứng dụng thực tế

- Ứng dụng thực tế: Xử lý trạng thái trong Compose để thay đổi giao diện khi nhấn nút.
- Ví dụ: Tạo nút đếm số lần nhấn.

```
@Composable
fun CounterButton() {
    var count by remember { mutableStateOf(0) } // Biến trạng thái đếm số lần nhấn

    Button(onClick = { count++ }) { // Khi bấm nút, tăng biến count
        Text("Clicked $count times") // Hiển thị số lần nhấn
    }
}
```

# Column trong Jetpack Compose - Giới thiệu

- Column là thành phần giúp xếp các phần tử theo chiều dọc.
- Tính năng: Đơn giản, dễ sử dụng, tự động căn chỉnh các phần tử theo chiều dọc.
- Ví dụ:

```
@Composable
 fun SimpleColumn() {
     Column {
         Text("Item 1")
         Text("Item 2")
     }
 }
```

# Column trong Jetpack Compose - Cách sử dụng

Sử dụng Column: Sắp xếp các thành phần UI theo chiều dọc

```
@Composable
fun StyledColumn() {
    Column(
        modifier = Modifier.padding(16.dp),
        verticalArrangement = Arrangement.spacedBy(8.dp)
    ) {
        Text("Item 1")
        Text("Item 2")
        Text("Item 3")
    } }
```

# Column trong Jetpack Compose - Ứng dụng thực tế

- Ứng dụng thực tế: Dùng Column để tạo danh sách động với khoảng cách đều.
- Ví dụ: Xử lý danh sách với một số lượng phần tử thay đổi động.

```
@Composable
 fun DynamicColumn(items: List<String>) {
     Column {
         items.forEach { item ->
             Text(item)
         }
     }
 }
```

# Row trong Jetpack Compose - Giới thiệu

- Row là thành phần sắp xếp các phần tử UI theo chiều ngang.
- Tính năng: Đơn giản, linh hoạt để sắp xếp các thành phần ngang.
- Ví dụ:

```
@Composable
 fun SimpleRow() {
     Row {
         Text("Item A")
         Text("Item B")
     }
 }
```

# Row trong Jetpack Compose - Cách sử dụng

- Sử dụng Row: Để sắp xếp các thành phần UI theo chiều ngang.
- Ví dụ:

```
@Composable
 fun StyledRow() {
    Row(
        modifier = Modifier.fillMaxWidth(),
        horizontalArrangement = Arrangement.SpaceBetween
    ) {
        Text("Start")
        Text("End")
    } }
```

# Row trong Jetpack Compose - Ứng dụng thực tế

- Ứng dụng thực tế: Sử dụng Row để tạo layout ngang với căn chỉnh hợp lý.
- Ví dụ: Cải thiện giao diện người dùng với các thành phần căn chỉnh trong Row.

# Box trong Jetpack Compose - Giới thiệu

- Box: Là thành phần để xếp các phần tử lên nhau, cho phép căn chỉnh các phần tử trong không gian chung.
- Tính năng: Dễ dàng tạo giao diện lớp chồng như ảnh nền, văn bản, nút bấm, v.v.
- Ví dụ:

```
@Composable
 fun SimpleBox() {
     Box {
         Text("Layer 1")
         Text("Layer 2", modifier = Modifier.align(Alignment.Center))
     }
 }
```

# Box trong Jetpack Compose - Cách sử dụng

- Sử dụng Box: Tạo các phần tử chồng lên nhau, thay đổi vị trí bằng Modifier.align().
- Ví dụ:

```
@Composable
 fun PositionedBox() {
     Box(modifier = Modifier.size(100.dp)) {
         Text("Background", modifier = Modifier.align(Alignment.Center))
         Text("Foreground", modifier = Modifier.align(Alignment.BottomEnd))
     }
 }
```

# Box trong Jetpack Compose - Ứng dụng thực tế

- Ứng dụng thực tế: Tạo giao diện hình ảnh chồng lên nhau, như ảnh nền và các nút bấm.
- Ví dụ: Sử dụng Box để tạo giao diện màn hình chính với ảnh nền và văn bản.

# Modifier trong Jetpack Compose - Giới thiệu

- Modifier: Dùng để tùy chỉnh, thay đổi giao diện của các phần tử trong Compose.
- Tính năng: Căn chỉnh, padding, màu sắc, v.v.
- Ví dụ:

```
@Composable
  fun MyModifiedText() {
      Text("Hello World!", modifier = Modifier.padding(16.dp))
  }
```

# Modifier trong Jetpack Compose - Cách sử dụng

- Sử dụng Modifier: Cung cấp cách thức để tùy chỉnh các thành phần UI.
- Ví dụ: Thêm padding, background, hoặc căn chỉnh.

```
@Composable
 fun ModifiedButton() {
     Button(onClick = { /* Handle Click */ }, modifier = Modifier.padding(16.dp)) {
         Text("Click Me")
     }
 }
```

# State trong Jetpack Compose - Giới thiệu

- State: Dùng để lưu trữ dữ liệu mà UI cần phản hồi khi thay đổi.
- Tính năng: Quản lý trạng thái ứng dụng dễ dàng.
- Ví dụ:

```
@Composable
  fun Counter() {
      var count by remember { mutableStateOf(0) }
      Column {
          Text("Count: $count")
          Button(onClick = { count++ }) {
              Text("Increase")
          } } }
```

# State trong Jetpack Compose - Cách sử dụng

- Sử dụng State: Dùng remember và mutableStateOf để lưu và quản lý trạng thái.
- Ví dụ: Cập nhật UI khi trạng thái thay đổi.

```
@Composable
 fun TextWithState() {
     var text by remember { mutableStateOf("Hello") }
     Text(text = text)
     Button(onClick = { text = "Changed!" }) {
         Text("Change Text")
     }
 }
```

# State trong Jetpack Compose - Ứng dụng thực tế

- Ứng dụng thực tế: Quản lý các thành phần động trong UI, như thay đổi nội dung văn bản hoặc hình ảnh sau khi nhấn nút.
- Ví dụ: Quản lý sự thay đổi trạng thái của nút hoặc danh sách.

# Tương thích giữa Compose và Views

- **ComposeView:** Cho phép sử dụng Views truyền thống trong ứng dụng Compose.

  - Ví dụ: Tích hợp TextView vào Compose.

```
val textView = TextView(context)
textView.text = "Hello from View"
ComposeView(context).setContent { Greeting("Compose View!") }
```

- **Tích hợp Compose vào Views truyền thống:** Dùng Compose trong ứng dụng cũ mà không cần thay đổi toàn bộ giao diện.

# Tích hợp Jetpack Compose vào ứng dụng cũ

- Tích hợp Compose vào View truyền thống: Sử dụng ComposeView để nhúng Compose vào ứng dụng Android hiện có.
- Ví dụ: Sử dụng Compose để thay thế hoặc bổ sung các Views truyền thống.

```
val composeView = ComposeView(context)

  composeView.setContent { Greeting("Compose in View") }
```

# Bài tập thực hành

- Tạo màn hình chào mừng:Viết một hàm @Composable hiển thị thông điệp "Chào mừng đến với Jetpack Compose!" bằng Text().
- Tạo nút bấm thay đổi văn bản:Viết một ứng dụng đơn giản với một Button và một Text.Khi nhấn vào Button, nội dung của Text thay đổi.
- Thiết kế giao diện danh sách với Column:Tạo một danh sách gồm 5 mục sử dụng Column.Hiển thị mỗi mục bằng Text().
- Tùy chỉnh giao diện bằng Modifier:Viết một @Composable sử dụng Modifier để thay đổi padding, background, và căn giữa văn bản trên màn hình.

# Câu hỏi thảo luận

- Khi nào nên sử dụng Jetpack Compose thay vì XML truyền thống?
- Lợi ích của lập trình giao diện khai báo (Declarative UI) là gì?
- Modifier giúp ích gì trong việc tùy chỉnh giao diện trong Compose?
- Làm thế nào để quản lý state hiệu quả trong Jetpack Compose?
- Có thể kết hợp Jetpack Compose với giao diện XML truyền thống không? Nếu có, bằng cách nào?

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

# *Thank you!*

THÁI NGUYÊN, THÁNG 10 NĂM 2024